Chapter 1

Membership Function Generator

1.1 Code

```
function out = MembershipFunctions(x, n, Type)
    % Minimal, same behavior; tiny cleanups and safety for n=1.

    if nargin < 3 || isempty(Type)
        Type = 'trimf';
    end
    Type = lower(Type);

    xmin = min(x);
    xmax = max(x);

    c = linspace(xmin, xmax, n);   % Centers
    out = cell(n, 2);

    % Use a guarded denominator so n=1 doesn't divide by zero
    denom = max(n - 1, 1);

    switch Type
        case 'trimf'
            dx = (xmax - xmin) / denom;   % spacing/half-width
            for i = 1:n
                out{i,1} = 'trimf';
                out{i,2} = c(i) + [-dx 0 dx];
            end

        case 'gaussmf'
            sigma = 0.5 * (xmax - xmin) / denom;
            for i = 1:n
                out{i,1} = 'gaussmf';
                out{i,2} = [sigma c(i)];
            end
    end
end
```

Listing 1.1: `MembershipFunctions.m`

1.2 Explanation

Purpose. `MembershipFunctions` builds a bank of n evenly spaced fuzzy membership functions over the range of the data in `x`. It returns a cell array `out` of size $n \times 2$, where each row is {*type*, *parameters*}. For triangles (`'trimf'`), parameters are $[\,\ell,\ c,\ r\,]$ (left, center, right). For Gaussians (`'gaussmf'`), parameters are $[\,\sigma,\ c\,]$.

Walk-through.

`function out = ...` Declares the function. Output `out` is a cell array.

`if nargin < 3 ...` Defaults `Type` to `'trimf'` if not provided/empty.

`Type = lower(Type);` Case-insensitive handling of the MF type.

`xmin = min(x); xmax = max(x);` Compute the data bounds $[x_{\min}, x_{\max}]$.

`c = linspace(xmin, xmax, n);` Create n evenly spaced centers across the range.

`out = cell(n,2);` Preallocate: each row stores {*type*, *params*}.

`denom = max(n-1, 1);` Guard against division by zero when $n = 1$.

`switch Type` Branch by MF family:

`'trimf':` 1. `dx = (xmax - xmin) / denom;` sets half-width (spacing for $n > 1$).
2. Loop stores `'trimf'` and parameters $[c(i) - dx,\ c(i),\ c(i) + dx]$.

`'gaussmf':` 1. `sigma = 0.5 * (xmax - xmin) / denom;` shared spread ensuring overlap.
2. Loop stores `'gaussmf'` and parameters $[\sigma,\ c(i)]$.

Output format.

$$\texttt{out} = \begin{bmatrix} \{\texttt{type\_1}, \texttt{params\_1}\} \\ \vdots \\ \{\texttt{type\_n}, \texttt{params\_n}\} \end{bmatrix},$$

where `type_i` $\in$ {`'trimf'`, `'gaussmf'`} and `params_i` is $[\ell, c, r]$ for triangles or $[\sigma, c]$ for Gaussians.

Chapter 2

Lookup Table FIS Generator

2.1 Code

```
function fis = LookupTableFIS(x, y, nmf, mftype)
% LookupTableFIS
%   Builds a Mamdani FIS from (x,y) via a lookup-table style rule base.
%   - x: [N x nx] inputs (only ranges and MF activations are used)
%   - y: [N x 1] output samples
%   - nmf: 1x(nx+1) counts of MFs per input and output (default 3)
%   - mftype: 1x(nx+1) cell of MF types for each input and output
%             (default 'gaussmf')
%
%   Returns: mamfis object.

    % ---- Basic sizes ----
    nx = size(x, 2);              % number of inputs

    % ---- Defaults ----
    if nargin < 3 || isempty(nmf)
        nmf = 3 * ones(1, nx + 1);
    end
    if isscalar(nmf)
        nmf = repmat(nmf, 1, nx + 1);
    end
    if numel(nmf) ~= nx + 1
        % pad or trim to length nx+1
        nmf = nmf(:).';
        if numel(nmf) < nx + 1
            nmf = [nmf, repmat(nmf(end), 1, nx + 1 - numel(nmf))];
        else
            nmf = nmf(1:nx+1);
        end
    end

    if nargin < 4 || isempty(mftype)
        mftype = repmat({'gaussmf'}, 1, nx + 1);
    elseif ischar(mftype)
        mftype = repmat({mftype}, 1, nx + 1);
```

```
    elseif numel(mftype) ~= nx + 1
        if numel(mftype) < nx + 1
            fill = repmat(mftype{end}, 1, nx + 1 - numel(mftype));
            mftype = [mftype, fill];
        else
            mftype = mftype(1:nx+1);
        end
    end

    % ---- Create MFs (inputs A{i}, output B) ----
    A = cell(nx, 1);
    for i = 1:nx
        A{i} = CreateMembershipFunctions(x(:, i), nmf(i), mftype{i});
    end
    B = CreateMembershipFunctions(y, nmf(end), mftype{end});

    % ---- Prepare rules container ----
    if nx > 1
        Ssize = nmf(1:end-1);
    else
        Ssize = [nmf(1), 1];
    end
    S = cell(Ssize);
    S = S(:);

    nRules = numel(S);
    for r = 1:nRules
        S{r} = zeros(1, nmf(end));
    end

    % ---- Calculate rule scores ----
    XIND = zeros(nRules, nx);
    for r = 1:nRules
        xind = cell(1, nx);
        [xind{1:nx}] = ind2sub(Ssize, r);
        xind = cell2mat(xind);
        XIND(r, :) = xind;

        for yind = 1:nmf(end)
            bmf    = B{yind, 1};
            bparam = B{yind, 2};

            s = feval(bmf, y, bparam);

            for i = 1:nx
                amf   = A{i}{xind(i), 1};
                aparm = A{i}{xind(i), 2};
                s = s .* feval(amf, x(:, i), aparm);
            end

            s = sum(s);
            S{r}(yind) = s;
        end
    end
```

```
90
91      % ---- Keep only non-zero rules ----
92      R    = zeros(nRules, 1);
93      keep = true(size(S));
94      for r = 1:nRules
95          [Smax, R(r)] = max(S{r});
96          if Smax == 0
97              keep(r) = false;
98          end
99      end
100
101     Rules = [XIND, R];
102     Rules(:, end+1) = 1; % weight
103     Rules(:, end+1) = 1; % AND = 1
104     Rules = Rules(keep, :);
105
106     % ---- Build FIS ----
107     fis = mamfis('Name', 'Lookup Table FIS');
108
109     % Inputs
110     for i = 1:nx
111         xrange = [min(x(:, i)), max(x(:, i))];
112         fis = addInput(fis, xrange, 'Name', ['x' num2str(i)]);
113         for j = 1:nmf(i)
114             fis = addMF(fis, ['x' num2str(i)], A{i}{j, 1}, A{i}{j, 2}, ...
115                 'Name', ['A(' num2str(i) ',' num2str(j) ')']);
116         end
117     end
118
119     % Output
120     yrange = [min(y), max(y)];
121     fis = addOutput(fis, yrange, 'Name', 'y');
122     for bi = 1:nmf(end)
123         fis = addMF(fis, 'y', B{bi, 1}, B{bi, 2}, 'Name', ['B' num2str(bi)
            ]);
124     end
125
126     % Rules
127     fis = addrule(fis, Rules);
128 end
```

Listing 2.1: `LookupTableFIS.m`

2.2 Detailed Explanation

Purpose. This function constructs a complete **Mamdani-type fuzzy inference system (FIS)** using a *lookup-table* approach. Given:

- Input data matrix $\mathbf{x} \in \mathbb{R}^{N \times n_x}$
- Output vector $\mathbf{y} \in \mathbb{R}^N$

- Number of membership functions per variable (`nmf`)
- Type of membership functions (`mftype`)

it:

1. Creates fuzzy membership functions for each input and output variable.
2. Computes all possible combinations of input MFs (rules).
3. Scores each rule based on data activation.
4. Keeps only rules with non-zero support.
5. Builds and returns a complete `mamfis` object.

Step-by-Step Walkthrough.

`nx = size(x,2);` Counts the number of inputs n_x.

Defaults for `nmf`, `mftype`: - If not given, each input and the output gets 3 MFs. - If scalar, it is replicated. - If length mismatch, it is padded or trimmed. - `mftype` defaults to `'gaussmf'`.

MF creation: For each input i, calls `CreateMembershipFunctions` to get nmf(i) MFs. For output, same process.

Rule matrix preparation: Creates a cell array **S** where each element will store the accumulated firing strength for each output MF for that input combination.

Rule scoring: For each possible input MF combination (r):

1. Converts linear index r to subscripts ($xind$) — MF index for each input.
2. For each output MF index ($yind$):
   - Evaluate that output MF on all y samples (s).
   - For each input i, multiply s by the degree of activation of the selected MF for that input on $x(:, i)$. (This is the fuzzy `AND` operation using the product t-norm.)
   - Sum s across all samples — producing the *rule score*.

Rule pruning: For each rule, find the output MF with the highest score (`max`). Remove rules with zero score (`keep` mask).

FIS building: - Create empty Mamdani FIS. - Add inputs with their ranges and MFs. - Add output with range and MFs. - Add surviving rules with weight 1 and `AND` connective.

Why it works. This method converts your raw numerical data into a complete fuzzy model by:

- Systematically covering the input space with evenly spaced MFs.
- Creating a rule for every possible MF combination.
- Using actual dataset activations to assign rules to specific output MFs.
- Pruning unused rules to keep the rule base compact.

Output. The result is a ready-to-use `mamfis` object, which can be directly evaluated with `evalfis` for prediction or used for further tuning.

[12pt]report

[utf8]inputenc amsmath, amssymb graphicx float hyperref listings xcolor caption subcaption geometry booktabs enumitem tcolorbox margin=1in

Chapter 3

Mackey–Glass Time-Series Generation (MATLAB)

3.1 Overview

This section generates a synthetic nonlinear time series using the classical Mackey–Glass delay differential system, then plots the result. The discrete update we use is:

$$x_{t+1} \;=\; x_t \;+\; \Delta t \left(\frac{0.2\, x_{t-\tau}}{1 + x_{t-\tau}^{10}} \;-\; 0.1\, x_t \right),$$

where τ is the delay (in time steps) and Δt is the integration step.

What this script does.

- Initializes a history buffer long enough to hold the delay τ.
- Iteratively updates the state using the Mackey–Glass equation.
- Discards the pre-history so the output x has exactly n samples.
- Plots $x(t)$ for visual inspection.

3.2 MATLAB Code

```
%% Setup
clc
clear
close all

%% Generate Data (Mackey Glass)
tau = 30;       % Delay
dt  = 1;        % Step
n   = 600;      % Number of samples
x0  = 0.8;      % x(0)
```

```
11
12 tau_steps = tau / dt;
13 x = zeros(n + tau_steps - 1, 1);
14 x(1:tau_steps) = x0;
15
16 for t = 0:n-2
17     i     = t + tau_steps;
18     x_tau = x(i + 1 - tau_steps);
19     dx    = 0.2 * x_tau / (1 + x_tau^10) - 0.1 * x(i);
20     x(i+1)= x(i) + dt * dx;
21 end
22
23 x = x(tau_steps:end);
24 t = (0:n-1)';
25
26 figure
27 plot(t, x, 'LineWidth', 1.2)
28 title('Mackey Glass')
29 xlabel('t')
30 ylabel('x(t)')
31 grid on
```

Listing 3.1: Generating and plotting the Mackey–Glass time series

3.3 Generated Plot

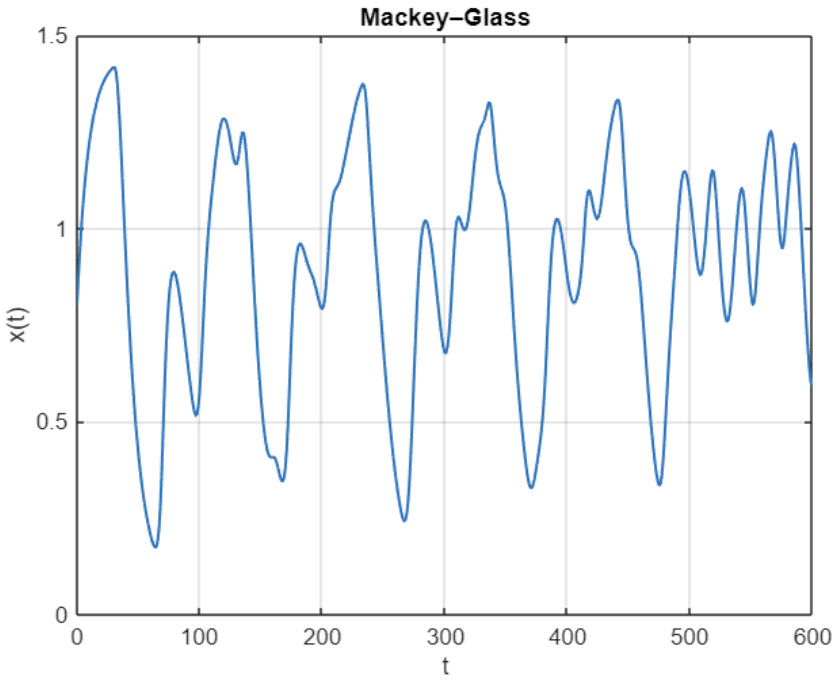

Figure 3.1: Mackey–Glass time series generated by Listing 3.1.

3.4 Explicit Explanation

Variables and Parameters

- `tau` ($\tau = 30$): delay horizon in time units.
- `dt` ($\Delta t = 1$): discrete integration step.
- `n` ($n = 600$): number of samples to *keep* in the final series.
- `x0` ($x(0) = 0.8$): initial condition used to seed the history.

Initialization and History Buffer

Because the dynamics depend on a delayed state $x_{t-\tau}$, we must keep a "pre-history" of length τ. The array

$$\mathtt{x} \in \mathbb{R}^{(n+\tau-1)\times 1}$$

is initialized to zeros and the first τ entries are set to `x0`. This lets the first $n-1$ updates access $x_{t-\tau}$ safely.

Main Update Loop

For $t = 0, 1, \ldots, n-2$:

1. Compute the absolute index $i = t + \tau\_steps$ (the current position in the buffer).
2. Fetch delayed state $x\_\tau = x(i + 1 - \tau\_steps) = x_{t+1-\tau}$.
3. Compute the Mackey–Glass right-hand side:

$$\dot{x}_t = 0.2\,\frac{x\_\tau}{1 + x\_\tau^{10}} - 0.1\,x(i).$$

4. Take an explicit Euler step:

$$x(i+1) = x(i) + \mathtt{dt} \cdot \dot{x}_t.$$

This is a straightforward forward-Euler discretization, adequate here for demonstration and for $\Delta t = 1$. For smaller Δt, you'd obtain a smoother trajectory (at increased computation).

Postprocessing

After the loop, the first τ entries are discarded:

$$\mathtt{x} = \mathtt{x}(\tau\_steps\colon \mathrm{end}),$$

so the returned series has exactly n samples. We then create a time vector $\mathtt{t} = (0{:}n-1)'$ and plot x versus t.

Complexity and Stability

- **Time complexity:** $O(n)$ operations and memory $O(n + \tau)$.
- **Numerical notes:** Forward Euler is conditionally stable. For stiff or highly delayed settings, consider a smaller Δt or a higher-order integrator.

Practical Tips

- If you modify τ or Δt, ensure `tau_steps` $= \tau/\Delta t$ is an integer, or adjust the indexing accordingly.
- To reproduce plots exactly across runs, keep the same `x0`. To randomize starts, set `x(1:tau_steps)=rand(...)` and fix a random seed.

3.5 FIS Model Training and Evaluation (Model 1)

3.5.1 MATLAB Code

```
%% Preprocess Data (Build Regressors)
% Predict y(t) from past 4 samples
y  = x(5:end);
x1 = x(4:end-1);
x2 = x(3:end-2);
x3 = x(2:end-3);
x4 = x(1:end-4);

X  = [x1 x2 x3 x4];

%% Train/Test Split
nTrain   = 300;
X_Train  = X(1:nTrain, :);
y_Train  = y(1:nTrain, :);
X_Test   = X(nTrain:end, :);
y_Test   = y(nTrain:end, :);

%% Normalize Inputs (mapminmax 0..1)
[X_Train_norm, settings] = mapminmax(X_Train', 0, 1);
X_Test_norm              = mapminmax('apply', X_Test', settings);
X_Train_norm             = X_Train_norm';
X_Test_norm              = X_Test_norm';

%% Model 1: Build FIS (7 MFs per var, trimf)
nmf1    = [7 7 7 7 7]; % 4 inputs + 1 output
mftype1 = {'trimf','trimf','trimf','trimf','trimf'};

fis1 = LookupTableFIS(X_Train_norm, y_Train, nmf1, mftype1);

%% Model 1: Evaluate
y_hat1_Train = evalfis(fis1, X_Train_norm);
y_hat1_Test  = evalfis(fis1, X_Test_norm);
```

```
33
34 %% Model 1: Plot
35 figure
36 subplot(2,1,1)
37 plot(y_Train, 'LineWidth', 1.0); hold on
38 plot(y_hat1_Train, 'LineWidth', 1.0)
39 title('Model 1 (7 MFs)     Train')
40 ylabel('x(t)')
41 xlabel('t')
42 grid on
43 legend('Actual','Predicted')
44
45 subplot(2,1,2)
46 plot(y_Test, 'LineWidth', 1.0); hold on
47 plot(y_hat1_Test, 'LineWidth', 1.0)
48 title('Model 1 (7 MFs)     Test')
49 ylabel('x(t)')
50 xlabel('t')
51 grid on
52 legend('Actual','Predicted')
```

Listing 3.2: Preprocessing, training, and evaluating FIS Model 1

3.5.2 Results

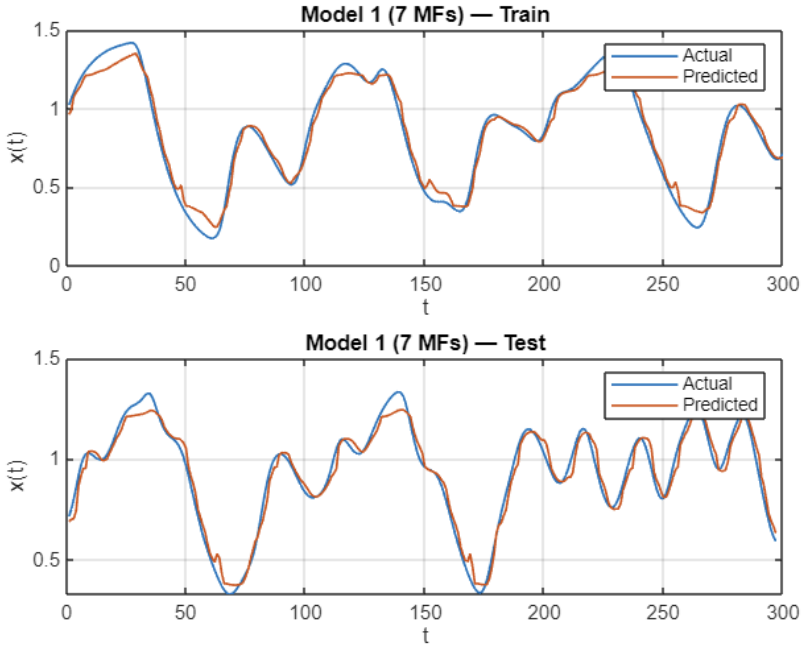

Figure 3.2: FIS Model 1 performance on training and test sets (7 MFs per input/output, triangular MFs).

3.5.3 Detailed Explanation

Preprocessing (lines 2–9). We create the regression matrix X and target vector y by using a **sliding window of four past samples** to predict the current output $y(t)$:

$$y(t) = x(t), \quad x_1 = x(t-1),\ x_2 = x(t-2),\ x_3 = x(t-3),\ x_4 = x(t-4).$$

This transforms the time series into a supervised learning dataset for the fuzzy model.

Train/Test Split (lines 11–16). The first $n_{\text{Train}} = 300$ samples go into the training set $(X_{\text{Train}}, y_{\text{Train}})$, while the remainder is used for testing. This split allows us to assess the model's ability to generalize.

Normalization (lines 18–21). We normalize all inputs to the range $[0, 1]$ using MATLAB's `mapminmax`:

$$X'_{\text{norm}} = \frac{X' - X_{\min}}{X_{\max} - X_{\min}}$$

The scaling parameters (`settings`) are computed from the training set and then applied to the test set to ensure consistent transformation.

Model Construction (lines 23–26). We specify:

- `nmf1` $= [7, 7, 7, 7, 7] \rightarrow$ 7 membership functions for each of 4 inputs and the output.
- `mftype1` $\rightarrow$ all triangular (`'trimf'`).

We then call `LookupTableFIS` to build the Mamdani FIS from the normalized training data. This function generates:

1. Membership functions for all variables.
2. All possible fuzzy rules from MF combinations.
3. Rule consequents determined from training data activation.

Model Evaluation (lines 28–29). We use `evalfis` to:

- Predict the training set outputs (y_hat,Train).
- Predict the test set outputs (y_hat,Test).

Visualization (lines 31–47). Two subplots are created:

1. **Top:** Actual vs predicted outputs on the training data.
2. **Bottom:** Actual vs predicted outputs on the test data.

The closeness of the predicted curve to the actual data indicates the accuracy of the fuzzy model.

Interpretation of Figure 3.2. The training subplot should show a very close match, indicating the FIS has learned the mapping from lagged inputs to $y(t)$. The test subplot reveals how well the model generalizes to unseen data. Discrepancies here can indicate overfitting or insufficient MF granularity.

3.6 FIS Model Training and Evaluation (Model 2)

3.6.1 MATLAB Code

```
%% Model 2: Build FIS (15 MFs per var, trimf)
nmf2    = [15 15 15 15 15];
mftype2 = {'trimf','trimf','trimf','trimf','trimf'};

fis2 = LookupTableFIS(X_Train_norm, y_Train, nmf2, mftype2);

%% Model 2: Evaluate
y_hat2_Train = evalfis(fis2, X_Train_norm);
y_hat2_Test  = evalfis(fis2, X_Test_norm);

%% Model 2: Plot
figure
subplot(2,1,1)
plot(y_Train, 'LineWidth', 1.0); hold on
plot(y_hat2_Train, 'LineWidth', 1.0)  % fixed: was y_hat1_Train
title('Model 2 (15 MFs)     Train')
ylabel('x(t)')
xlabel('t')
grid on
legend('Actual','Predicted')

subplot(2,1,2)
plot(y_Test, 'LineWidth', 1.0); hold on
plot(y_hat2_Test, 'LineWidth', 1.0)
title('Model 2 (15 MFs)     Test')
ylabel('x(t)')
xlabel('t')
grid on
legend('Actual','Predicted')
```

Listing 3.3: Training and evaluating FIS Model 2 (15 MFs per variable)

3.6.2 Results

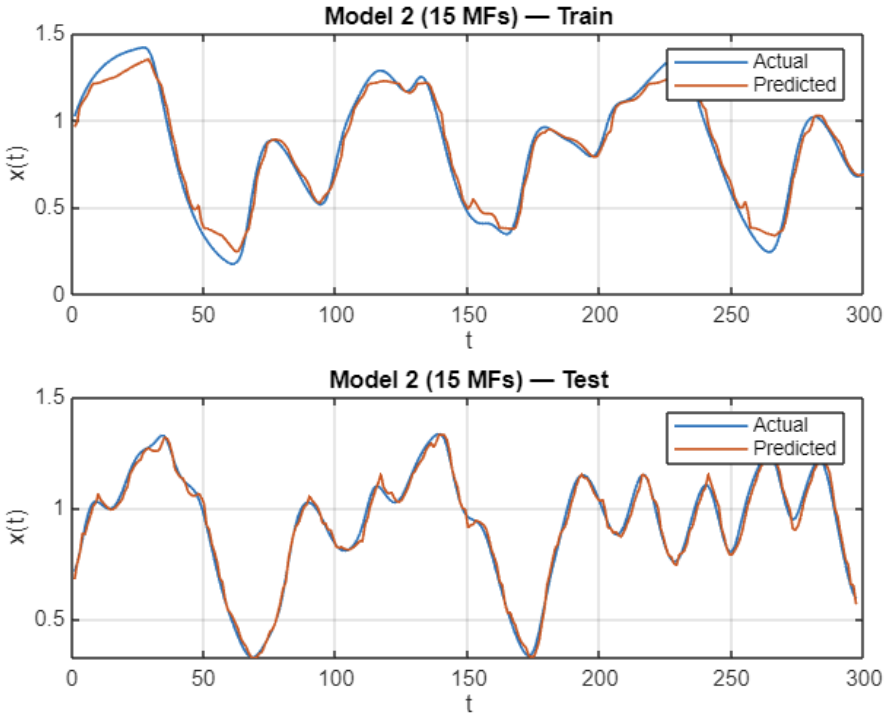

Figure 3.3: FIS Model 2 performance with 15 triangular membership functions per variable.

3.6.3 Detailed Explanation

Why increase to 15 MFs? Compared to Model 1 (7 MFs), Model 2 substantially increases the input–output partition granularity. This can capture finer nonlinearities in the Mackey–Glass mapping from lags $[x(t-1), \ldots, x(t-4)]$ to $y(t) = x(t)$. However, it also:

- Greatly expands the rule space: with 4 inputs, the *maximum* number of antecedent combinations is $15^4 = 50{,}625$ (before pruning).
- Increases computational cost for rule scoring and evaluation.
- Raises the risk of overfitting if training data are not sufficiently diverse.

FIS construction.

1. **MF specification:** `nmf2` assigns 15 triangular MFs to each of the 4 inputs and to the output.
2. **LookupTableFIS:** Builds input and output MFs, enumerates all input MF combinations, scores rule consequents against the training data (using product t-norm and sum aggregation over samples), and prunes rules with zero support.
3. **Resulting FIS:** A Mamdani `mamfis` ready for `evalfis`.

Evaluation protocol.

- **Training prediction:** $\hat{y}_{\text{Train}} = \texttt{evalfis}(\texttt{fis2}, X_{\text{Train\_norm}})$.
- **Test prediction:** $\hat{y}_{\text{Test}} = \texttt{evalfis}(\texttt{fis2}, X_{\text{Test\_norm}})$.
- Both sets use the same normalization settings learned from the training inputs (to avoid leakage).

Reading Figure 3.3.

1. **Train subplot:** If the predicted curve overlaps closely with the actual series, the rule base has sufficient expressive power for the training set.
2. **Test subplot:** This indicates generalization. If performance worsens compared to Model 1 (more error or phase lags), the model may be overfitting due to excessive partitioning.

Practical considerations.

- **Rule explosion:** Even with pruning, 15 MFs per input can lead to a very large rule base. Consider reducing MFs or using data-driven MF placement (e.g., clustering) if runtime grows too much.
- **Regularization levers:** Fewer MFs, different MF shapes (e.g., Gaussian), or changing the t-norm/aggregation can improve stability.
- **Fair comparison:** Always keep the same train/test split and normalization when comparing Model 1 vs. Model 2.